

=====

OPENAI BATCH API OVERBILLING REPORT

Date range covered: 2026-07-03 to 2026-07-04 (UTC)

Prepared: 2026-07-04

=====

PURPOSE

This document records, with evidence pulled directly from OpenAI's own Administration (usage & costs) API, a series of Batch API runs in which the account was charged for a large volume of model executions that produced NO returned output. It is written so it can be adapted into a refund request. It is a factual evidence record, not the request itself.

All figures below are taken from:

- GET /v1/batches/{id} (per-batch request_counts + usage)
- GET /v1/organization/usage/completions (billed tokens & model-request counts)
- GET /v1/organization/costs (authoritative dollar charges)

and cross-checked against the batch output/error files.

1. ACCOUNT / IDENTIFIERS

Organization ID : org-BH1YvLA20I41CevsyGEb5jp7 (name: "Personal")
Project ID : proj_eNdWq3WgIuAZZUILLVZLgU35 (name: "Frank Stewart Bedouin Fieldnotes")
Model : gpt-5.5-2026-04-23 (reasoning effort: high; tool: Code Interpreter)
Endpoint : /v1/responses via the Batch API (completion_window 24h)

2. EXECUTIVE SUMMARY (THE CLAIM IN BRIEF)

Across three Batch API jobs in this window:

- Requests SUBMITTED by us 43
- Requests that COMPLETED (returned output) ... 8
- MODEL EXECUTIONS actually billed 89 (2.1x the requests submitted)
- Batch API dollars billed \$43.58
- Value of the 8 completed requests \$2.38
- => Billed for work that produced NO output ... ~\$41.20

OpenAI's stated Batch API billing policy (OpenAI support, OpenAI Developer Community, thread "Batch API billing for failed requests (504 Timeout)":
"billing is based on token usage from completed requests"; failed requests that appear in the error file "without response or usage data are excluded from billing."

The charges below are inconsistent with that policy: the account was billed for 4.81M input and 2.14M output tokens, while the 8 requests that actually returned output account for only 0.24M input and 0.12M output tokens. The remaining ~4.57M input / ~2.02M output tokens were consumed by requests that FAILED and by undisclosed internal retries of those failed requests, and returned nothing.

3. THE THREE BATCHES

BATCH-1 (initial run)

batch_id : batch_6a482b22b14481909cb74e9a61327845
created : 2026-07-03 21:35 UTC status: completed
submitted 20 | completed 3 | failed 17
failure type : 17 x HTTP 429 insufficient_quota
billed : 25 model executions | 1,273,853 in / 588,591 out (~\$11.97)

completed-only tokens (batch.usage): 93,829 in / 49,765 out (~\$0.98)

BATCH-2 (retry of the 17 failures from Batch-1)

batch_id : batch_6a4858b31da081909c6abab3c42a7e74
created : 2026-07-04 00:49 UTC status: completed
submitted 17 | completed 3 | failed 14
failure type : 12 x HTTP 429 insufficient_quota ; 2 x HTTP 400 "issue with your request"
billed : 37 model executions | 2,082,425 in / 876,053 out (~\$18.35)
completed-only tokens (batch.usage): 83,580 in / 40,024 out (~\$0.81)

BATCH-3 (fresh 6-scan run, on a funded balance)

batch_id : batch_6a48a41f79348190a4b9921d50adad51
created : 2026-07-04 06:11 UTC status: cancelled (cancelled by us to stop the burn)
submitted 6 | completed 2 | failed 3 (+1 cancelled in-flight)
failure type : 3 x HTTP 400 "issue with your request" ; 1 x HTTP 503 server_is_overloaded
billed : 27 model executions | 1,454,322 in / 671,733 out (~\$13.71)
completed-only tokens (batch.usage): 58,818 in / 29,434 out (~\$0.59)

NOTE ON BATCH-3: 6 requests were submitted but 27 model executions were billed
-- a 4.5x multiplication with no user-side retries. This batch was submitted on
a comfortably funded balance, so its failures were NOT caused by low credit;
they were server-side errors (400 / 503) that were then retried internally.

4. KEY EVIDENCE

(A) EXECUTIONS FAR EXCEED REQUESTS SUBMITTED (undisclosed internal retries)

Requests submitted (all 3 batches) 43
Model executions billed 89
Ratio 2.07x (Batch-3 alone: 27 / 6 = 4.5x)
The Batch API documentation exposes no retry parameter and does not disclose
that failed requests are retried. The only explanation for 89 executions of
43 requests is undisclosed server-side retries, each one billed.

(B) BILLED TOKENS FAR EXCEED COMPLETED-REQUEST TOKENS

	INPUT tokens	OUTPUT tokens
Billed (usage API)	4,810,600	2,136,377
Completed requests	236,227	119,223
Billed for nothing	4,574,373	2,017,154

~95% of the input tokens and ~94% of the output tokens billed produced no
returned result.

(C) DOLLARS: BILLED vs. VALUE DELIVERED

Batch API billed (costs API) \$43.58
Value of the 8 completed requests \$2.38
Charged for zero-output work ~\$41.20

(D) FAILURES WERE NOT CAUSED BY MALFORMED INPUT ON OUR SIDE

- HTTP 503 "server_is_overloaded" is explicitly a server-side condition.
- HTTP 400 "There was an issue with your request" was returned with NO
error code and a generic message. The same 5 image files that triggered
it are identical in format (PNG), dimensions (~4050 x 6800 px) and size
range to files that SUCCEEDED in the same batches, so the input itself
was valid and accepted.
- HTTP 429 insufficient_quota in Batch-1/2 was itself a consequence of the
retry storm draining the balance mid-run; each retried execution was
billed before the balance check failed.

DIRECT PROOF THE 400 IS A FALSE (SERVER-SIDE) ERROR:

Scan FN-0006 failed with HTTP 400 "There was an issue with your request."
Please check your inputs and try again" in the Batch API (in both Batch-1
and Batch-3). The SAME file was then submitted via the SYNCHRONOUS
/v1/responses API with the IDENTICAL configuration (gpt-5.5, effort high,

Code Interpreter on, same prompt) on 2026-07-04 07:45 UTC. It COMPLETED SUCCESSFULLY on the first attempt: 39,807 input / 19,851 output tokens, returning a full transcription for ~\$0.79. The input was therefore valid and processable; the Batch API's 400 "check your inputs" response was a false, server-side failure -- and it was that false failure, retried internally, that generated the bulk of the zero-output charges.

(E) OPENAI'S OWN BILLING RECORD CONFIRMS THE TOKEN COUNTS

From GET /v1/organization/costs (line items, gpt-5.5-2026-04-23, Batch API):

2026-07-03	input	1,250,557 tok	\$3.13
2026-07-03	output	588,591 tok	\$8.83
2026-07-03	cached input	23,296 tok	\$0.01
2026-07-04	input	3,335,275 tok	\$8.34
2026-07-04	output	1,547,786 tok	\$23.22
2026-07-04	cached input	201,472 tok	\$0.05

Batch API total			\$43.58

(F) DOCUMENTED PRECEDENT (OpenAI Developer Community)

A materially identical case -- "Batch API charged \$212 for gpt-5.4-pro with zero completed results" -- reported 34 requests submitted but 98 processed internally (~3x), 2.07M output tokens never returned, and \$212 billed for zero usable results. OpenAI investigated (Case ID 07830071), CONFIRMED the problem, and issued a full refund. The same signature (internal ~3x-4x retries billed on failed reasoning-model batch requests) is present here.

5. WHAT IS *NOT* BEING CLAIMED (for transparency)

Separately in this window the account also incurred ~\$17.78 of NON-batch (synchronous) gpt-5.5 charges (2026-07-03: 921,154 input \$4.61 + 439,135 output \$13.17). Those were interactive single-image test runs that COMPLETED and returned output, and are NOT part of this claim. They are listed here only so the batch figures can be cleanly separated from the account's total spend for the day (grand total for the window, all line items: \$61.35).

5b. COMPLETENESS CHECK (entire account history audited)

To confirm this report captures ALL affected charges, every Batch API job ever created on the account was reviewed (GET /v1/batches, full pagination) and the entire Batch-API cost history was pulled (GET /v1/organization/costs, grouped by line item over the account's lifetime):

- Total Batch API jobs ever created 8
- Of these, 5 earlier test jobs (2026-07-01 to 2026-07-02, 3 requests each) failed / were cancelled / expired BEFORE consuming billable tokens and incurred \$0.00 in Batch API charges.
- The account's ENTIRE Batch-API spend across all history is \$43.57, occurring on only two days -- 2026-07-03 (\$11.96) and 2026-07-04 (\$31.61) -- which are exactly the three batches documented in this report (\$43.58; matches to the cent).

There are therefore no additional or earlier batch cycles with the same problem outside the scope of this report.

6. BASIS FOR RESOLUTION

Per OpenAI's stated policy that only completed requests are billable, the refundable amount is the Batch API charges less the value of the 8 completed

requests:

```
Batch API billed ..... $43.58
Completed-request value .. $2.38
-----
Refund basis ..... ~$41.20
```

Supporting artifacts available on request: the three batch objects (with request_counts and usage), the output and error JSONL files for each batch, and the raw /v1/organization/usage and /v1/organization/costs responses backing every figure above.

7. APPENDIX -- HOW THE FIGURES WERE OBTAINED (reproducible)

Per-batch counts & completed-only usage:

GET https://api.openai.com/v1/batches/{batch_id}

Billed tokens & model-execution counts (grouped by batch, hourly):

GET https://api.openai.com/v1/organization/usage/completions
?start_time=...&bucket_width=1h&group_by=batch

Authoritative dollar charges (grouped by line item, daily):

GET https://api.openai.com/v1/organization/costs
?start_time=...&bucket_width=1d&group_by=line_item

Failure reasons per request:

the error_file_id JSONL of each batch (status_code + error.message)

Rates applied (gpt-5.5, Batch API = 50% off standard):

input \$2.50 / 1M tok , output \$15.00 / 1M tok

CONCLUSION / FINAL RECONCILIATION (added 2026-07-06)

Outcome of the claim, reconciled against GROUND-TRUTH billing data from the OpenAI dashboard (credit-grants table + invoice history) -- neither of which the API exposes. This supersedes earlier in-conversation estimates of the refund.

REFUND RECEIVED

- OpenAI issued ONE refund of \$25.98, as a credit grant dated 2026-07-04, against the \$41.20 originally claimed for the three incident batches.
- ~63% of the claim was honored. ~\$15.22 of the original claim is still unrefunded (plus later batch waste that was never part of the claim).
- Earlier balance-delta estimates suggested ~\$41 refunded; those were WRONG. The grants table is authoritative: the refund is \$25.98.

GROUND-TRUTH FINANCIAL RECONCILIATION (reconciles exactly)

```
Credit purchased (8 invoices, pre-tax) ..... $95.00
Refund credit (2026-07-04 grant) ..... $25.98
Total credit granted ..... $120.98
Total consumed by ALL API usage (net) ..... $45.00
Current balance (2026-07-06) ..... $75.98
  check: 95.00 + 25.98 - 45.00 = 75.98    OK
Out of pocket paid (incl. ~$4.92 tax/fees) .... $99.92
```

BATCH WASTE vs. REFUND

```
Gross batch billed (frozen token data) ..... $54.42
Value of batch pages that completed ..... $3.32
Batch waste (failed/retried, zero output) ..... $51.10
Refunded ..... $25.98 (~51% of the waste)
Unrefunded failed-attempt waste ..... ~$25.12
```

DATA-RELIABILITY NOTES (what to trust)

- Balance / credits: NOT available via API (endpoints return 404/403). The dashboard credit-grants table is the authoritative money ledger.
- Usage / cost API: it AGES. Execution counts and daily costs drop on re-query for older batches (Batch-3's execution count fell 27 -> 6). Do not trust it for history. All batch \$ figures here come from token data frozen when fresh.
- Token-based cost math OVERSTATES real charges when runs repeat, because prompt/image caching (cached input billed at 10%) is not reflected. The 59 repeated experiment runs are the main reason true net spend (\$45.00) is far below the ~\$75-100 the token math implied.

COST PROJECTION -- 1,200 FRESH SCANS (unique images, so no cross-scan caching)
 Synchronous only (actual sync runs ~\$1.00/scan) ~\$1,200
 Clean batch + sync fallback (watchdog + funded balance) . ~\$700
 Batch "as it actually happened" (storm waste extrapolated) ~\$2,200
 (cautionary figure, NOT a forecast -- assumes the one-time low-balance
 retry storm repeats, which the watchdog + funding now prevent)

RECOMMENDATION

- Follow up with OpenAI support for the remaining ~\$15.22 of the \$41.20 claim, citing the \$25.98 grant already issued (2026-07-04).
- For production: keep balances well above the worst case and run the cost watchdog. The Batch API is unreliable on this workload -- scans that fail repeatedly in batch transcribe fine synchronously -- so budget for a synchronous fallback on a meaningful fraction of scans.

=====